WORKED SAMPLE · PREPARED FOR YOUR $250 REVIEW

Reviewing information-dense B2B data tables

A dashboard built around filtering, sorting, editing and comparing large tables fails in ways that are hard to see from the inside — every individual screen looks reasonable, but the workflow between them doesn't hold up. This is the framework I apply, and three examples of the kind of finding it produces.

Scope: dashboard + configuration page

Deliverable: prioritised issue list + recommendations

Format: annotated screens, severity-ranked

01 What the $250 review actually produces

Not a mood board, and not a redesign. The goal is to tell you *what to keep, what to fix, and in what order* before you ship — so that the refinement phase is execution rather than exploration.

ARTEFACT	WHAT'S IN IT
Annotated screen set	Every issue pinned to the exact element and screen it occurs on.
Severity-ranked issue list	Each issue rated HIGH blocks launch / MEDIUM costs trust or time / LOW polish. Sortable, so it can become a ticket queue directly.
Keep / fix / cut verdict	Explicit call on your in-progress redesign concept: which parts to protect, which to abandon, and why.
Prioritised next steps	A sequenced short list — usually 5–8 items — with rough effort, so you can scope the refinement phase honestly.
Open questions	The things I can't judge without knowing your users: who edits vs who reads, what "large" means in rows, what happens when a comparison is wrong.

02 The checklist I run against tables specifically

Density & scanability

- Can a row be parsed left-to-right, or does it need reading twice?
- Is the density user-controlled (row-height toggle, column visibility)?
- Do numeric columns align on the digit, and right-align?
- Are units and precision consistent down the column?

Filtering

- Is the active filter state always visible, or hidden behind a panel?
- Named filters vs raw queries — can a colleague reproduce yours?
- Are counts shown before applying, so the user isn't clicking into an empty result?
- Is "clear all" one action, and does it restore the previous state?

Sorting & comparison

- Is multi-column sort visible, and is the primary key unmistakable?
- Can a column be pinned while scrolling horizontally?
- For value comparison: is the delta shown, or must the user hold two rows in their head?
- Is sorting stable when values tie?

Editing

- Inline vs modal decided per field, not globally — long values want a modal, toggles don't.
- Is a failed save surfaced at the cell, not as a page-level toast?
- Is there undo, or at least an obvious confirmation for destructive edits?
- Does the row show who changed what, and when?

Bulk actions

- Is selection persistent across pages and after filtering?
- Is the selected count always on screen while the action bar is open?
- Can a selection be inverted or cleared in one click?

States

- Empty state distinguishes "no data yet" from "your filter matched nothing".
- Loading state preserves column widths — no layout jump on arrival.
- Partial-failure state: what is shown when 3 of 50 rows fail to load?

Configuration page

- Grouped by task, not by underlying data model.
- Destructive settings are separated and styled as such.
- Is there a preview or "what changes" summary before saving?
- Defaults are visible, and resetting is discoverable.

Accessibility & keyboard

- Focus is visible and ordered; the table is operable without a mouse.
- Contrast holds for muted text and disabled controls, not just body copy.
- Column headers are real headers, so screen readers announce the pair.

03 Three examples of the findings this produces

Illustrative patterns, drawn from how data-heavy tables typically fail. Not references to your app — I haven't seen it yet.

The filter state is invisible on the table screen HIGH

Filters live in a side panel. Once it's closed, the table shows a subset with no indication of why. A user returning after lunch, or a colleague opening a shared link, cannot tell a filtered view from a complete one — and will make decisions on a partial dataset.

Fix: persistent, removable filter chips above the table, with a total-vs-shown count. Cheap, and it removes a whole class of "the numbers are wrong" support tickets.

Comparison forces short-term memory HIGH

Two rows are compared by scrolling between them. For the primary user — someone deciding between values — this is the core task, and it currently depends on the user remembering the first row accurately.

Fix: a compare mode that pins the selected rows together and renders the delta explicitly. This is usually the single highest-value change in a comparison-driven table.

Editing behaves the same for every field type MEDIUM

All edits open the same modal. For a short enum or a boolean, that's three clicks and a context switch for something a single click could do; for a long free-text field, an inline editor is cramped and loses formatting.

Fix: match the editor to the field. Inline for short and bounded, modal for long or multi-part, and validation surfaced at the cell rather than as a page-level error.

04 How the two-phase engagement would run

Day 1	You give me access — a walkthrough video or a staging link both work. Plus 15 minutes of context: who uses the dashboard, and what decision they're making with it.
Day 2–3	Review pass. I work the checklist above against both the current design and the redesign concept, and write up findings as I go.
Day 4	Annotated screen set + severity-ranked issue list + keep/fix/cut verdict on the redesign concept + prioritised next steps. That's the $250.
Then	Optional refinement at $30–40/hr, scoped from the prioritised list — so you're agreeing to a shape and an estimate, not an open-ended block of hours.

One thing worth deciding before the review

You have both a current design and an in-progress redesign. The most useful question I can answer for you is not "which looks better" but **whether the redesign actually fixes the workflow problems, or only changes the surface**. That judgement is the reason to bring in an outside read, and it's what I'd weight the review toward.
